

Assignment 2: Coding Basics

Sarah Hall

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file <FirstLast>_A02_CodingBasics.Rmd (replacing <FirstLast> with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.  
#Generates a sequence of numbers from 1 to 55, increasing by 5s.  
#Sequence is named sequence_by_five  
sequence_by_five <- seq(1,55,5)
```

```
#2.  
  
mean(sequence_by_five) #calculates the mean
```

```
## [1] 26
```

```
median(sequence_by_five) #calculates the median
```

```
## [1] 26
```

```
#3.  
#evaluates if the mean is greater than the median, returns a boolean value  
mean(sequence_by_five) > median(sequence_by_five)
```

```
## [1] FALSE
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#creates three vectors each with four components
NameVector <- c("Sarah", "Jim", "Bob", "Joe") #Vector Type: character
ScoreVector <- c(97, 82, 89, 74) #Vector Type: numeric
ScholarshipVector <- c(TRUE, TRUE, FALSE, FALSE) #Vector Type: logic

#creates dataframe with three vectors
StudentInformation <- data.frame(NameVector, ScoreVector, ScholarshipVector)
#names columns in dataframe
names(StudentInformation) <- c("Student Name", "Test Score", "Scholarship Status")
```

9. QUESTION: How is this data frame different from a matrix?

Answer: This data frame stores data of all different types (logic, character, and numeric), while a matrix stores data that is all the same data type. Data frames can also have a variable number of rows and columns (in this example: 3 columns, 4 rows), while a matrix has fixed rows and columns.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

#10. Create a function using if...else

```
PassFail1 <- function(test_score) {
  if(test_score > 50){
    print("Pass")
  }
  else{
    print("Fail")
  }
}
```

#11. Create a function using ifelse()

```
PassFail2 <- function(test_score) {
  ifelse(test_score>50,"Pass","False")
}
```

```
}
```

```
#12a. Run the first function with the value 52.5
```

```
PassFail1(52.5)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5
```

```
PassFail2(52.5)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores
```

```
#PassFail1(ScoreVector) #function does not work
```

```
#13b. Run the second function with the vector of test scores
```

```
PassFail2(ScoreVector)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: `ifelse` worked. `ifelse` is designed for vectorized operations, while `if...else` is for single scale inputs. Vectorized operation means the functions loops through and operates on all elements of the vector automatically.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)